Feature Selection in Image Analysis using Evolutionary Algorithms

Diabetic Retinopathy – APTOS Dataset

Applying Genetic Algorithms and Particle Swarm Optimization to select the most relevant features from retinal images, improving classification performance while minimizing feature count.

EVOLUTIONARY COMPUTING | MEDICAL IMAGING | FEATURE SELECTION

Made with GAMMA

Introduction

Feature selection is a critical step in image analysis, especially in medical diagnosis such as diabetic retinopathy detection. High-dimensional image features can lead to overfitting, increased computational cost, and reduced model interpretability.

This project focuses on applying **Evolutionary Algorithms (EAs)** to select the most relevant features from retinal images (APTOS dataset), improving classification performance while minimizing feature count.

The Challenge

- Overfitting from high-dimensional features
- Increased computational cost
- Reduced model interpretability

The Solution

- Evolutionary Algorithms for feature selection
- APTOS 2019 retinal fundus dataset
- Multi-objective optimization approach

Made with GAMMA

Problem Formulation & Modeling

1

Problem Type

Multi-objective optimization: **maximize** classification accuracy and **minimize** number of selected features.

2

Solution Representation

Each candidate solution is a binary vector $X = [x_1, x_2, \ldots, x_n]$ where $x_i = 1$ means feature selected and $x_i = 0$ means not selected.

3

Fitness Function

$\text{Fitness} = \alpha \times \text{Accuracy} + (1 - \alpha) \times \text{Reduction}$, balancing performance and simplicity.

4

Dataset

APTOS 2019 Blindness Detection Dataset — retinal fundus images. Task: classify diabetic retinopathy severity. Preprocessing: resizing, normalization, and CNN feature extraction.

Made with GAMMA

System Architecture

The system follows a structured pipeline from raw retinal images to final classification evaluation, integrating deep learning feature extraction with evolutionary feature selection.

Input Images

Retinal fundus images

Preprocessing

Resize & normalize

Feature Extraction

Deep CNN (ResNet50)

Feature Selection

GA / PSO optimization

Classification

SVM, RF, KNN

Evaluation

Metrics & results

Made with GAMMA

Dataset Architecture & Deep Feature Extraction Pipeline

Our methodology begins with a robust dataset, processed through a meticulously engineered pipeline to prepare for feature selection. This involves initial standardization, advanced pixel normalization, and deep feature extraction using a pre-trained Convolutional Neural Network.

Dataset Overview

- **Dataset:** APTOS 2019 Blindness Detection Dataset
- **Data Type:** High-resolution retinal fundus images
- **Primary Task:** Multi-class classification of Diabetic Retinopathy severity

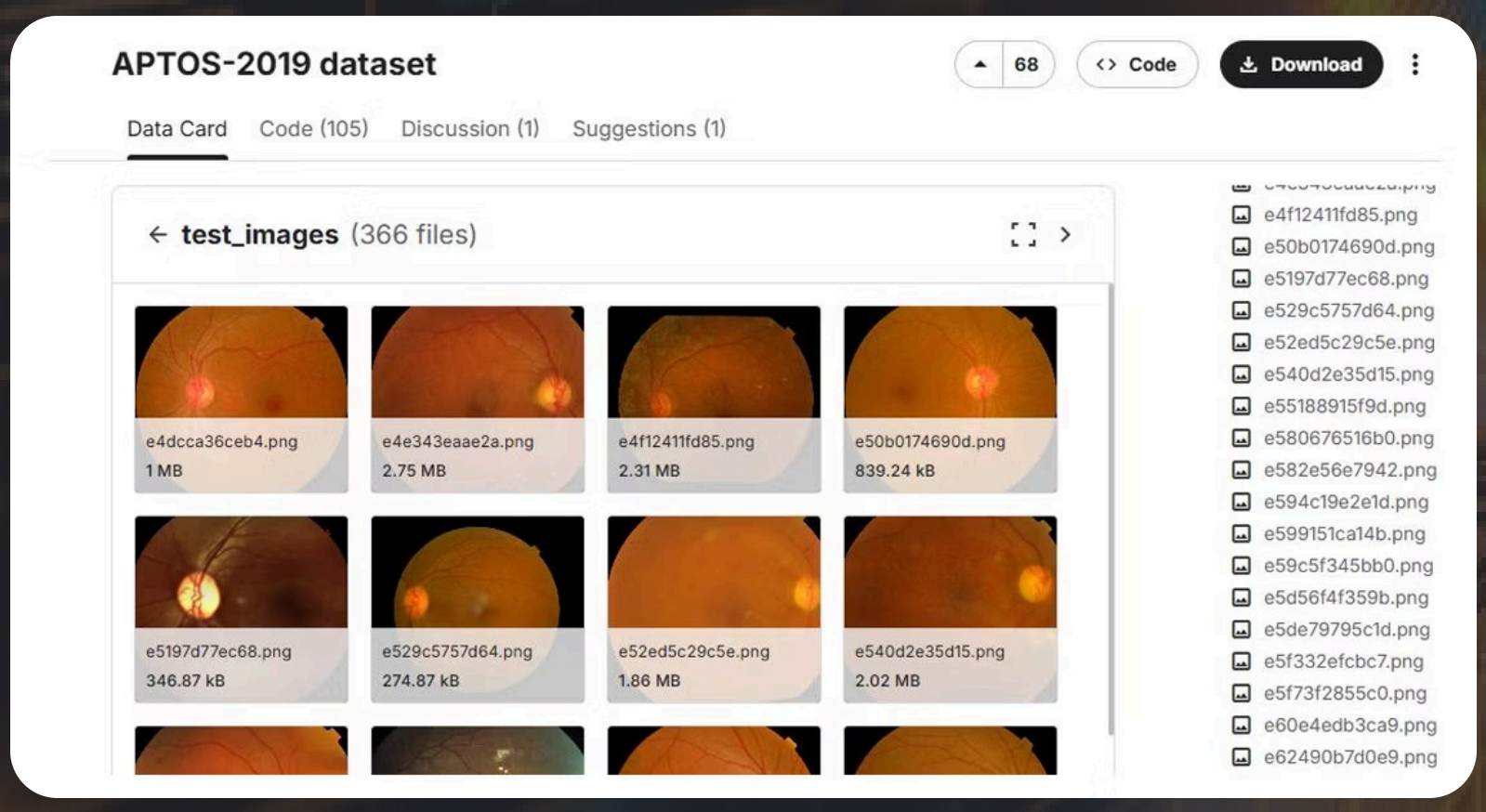
Preprocessing Pipeline

To ensure optimal performance and comparability across all samples, raw retinal images underwent several critical preprocessing stages. This rigorous preparation minimized variability and maximized data quality for subsequent analytical steps.

- **Standardization & Resizing:** Addressing the inherent variability in raw retinal images—arising from diverse lighting conditions, camera models, and potential artifacts—all images were rigorously standardized and resized to a uniform matrix dimension. This step is crucial for maintaining consistency throughout the neural network's input layers.
- **Pixel Normalization:** A statistical normalization procedure was applied to scale pixel intensity values into a consistent range. This technique is instrumental in accelerating mathematical convergence during neural network training and prevents issues such as gradient saturation, which can impede effective feature extraction.

Deep Feature Extraction (ResNet50)

Rather than relying on labor-intensive, manual feature engineering, a pre-trained **Convolutional Neural Network (CNN)**, specifically **ResNet50**, was deployed as an autonomous feature extractor. This sophisticated model processed the normalized retinal images, adeptly capturing complex, high-level spatial patterns and intricate morphological details pertinent to Diabetic Retinopathy detection.



The Curse of Dimensionality

The output from the ResNet50 model resulted in a massive, high-dimensional vector, comprising exactly **2048 distinct features per image**. This immense feature space, while rich in information, introduced a significant risk of model overfitting and substantial computational lag. Such high dimensionality necessitated the deployment of our advanced **Genetic Algorithm (GA)** and **Particle Swarm Optimization (PSO)** engines to mathematically reduce the feature count to the optimal **~400-600 range**, thereby enhancing model robustness and efficiency.

Made with GAMMA

Methodology

Feature Extraction

- Used a pre-trained CNN (ResNet50)
- Extracted high-level features from images
- Converted features into 1D vectors
- Stored processed data for reuse

Fitness Function & Optimization

- Implemented caching (memoization) to avoid recomputation
- Handles edge cases (e.g., zero features)
- Fitness balances performance and simplicity

Classification Models

Used multiple classifiers:

SVM

Support Vector Machine

RF

Random Forest

KNN

K-Nearest Neighbors

These models evaluate feature subsets inside the fitness function.

Made with GAMMA

Genetic Algorithm

Population

Binary vectors representing candidate feature subsets. Each individual encodes a unique selection of features.

Selection Methods

Tournament selection and Roulette Wheel selection choose the fittest individuals for reproduction.

Crossover Operators

One-point crossover and Uniform crossover combine parent solutions to explore new feature combinations.

Mutation Operators

Bit flip mutation and Random mutation introduce diversity. Replacement is generational.

Made with GAMMA

Particle Swarm Optimization (PSO)

Unlike GA, PSO maintains both a **position vector** (binary) and a **velocity vector** (continuous). Each particle represents a candidate feature subset $X = [x_1, x_2, \ldots, x_n]$.

Velocity Update

$$v_i^{t+1} = w \cdot v_i^t + c_1 \cdot r_1 \cdot (pbest_i - x_i^t) + c_2 \cdot r_2 \cdot (gbest - x_i^t)$$

- w: inertia weight
- c_1, c_2: cognitive & social coefficients
- r_1, r_2: random values in [0,1]
- $pbest$: personal best, $gbest$: global best

Binary Transformation

Velocities are converted using a sigmoid function:

$$S(v) = \frac{1}{1 + e^{-v}}$$

$$x_i = \begin{cases} 1 & \text{if rand} < S(v_i) \\ 0 & \text{otherwise} \end{cases}$$

01

Initialize swarm randomly

02

Evaluate fitness of each particle

03

Update personal best (pbest)

04

Update global best (gbest)

05

Update velocities and positions using sigmoid

06

Repeat until termination

Made with GAMMA

Experiments & Results

Comparison of GA and PSO across multiple runs with evaluation of selection methods, crossover techniques, and inertia configurations.

98.6%

GA Accuracy

Best classification accuracy achieved by Genetic Algorithm

0.91

GA Best Fitness

Combined objective score for GA

300

GA Features

Number of features selected by GA

97.7%

PSO Accuracy

Best classification accuracy achieved by PSO

0.92

PSO Best Fitness

Combined objective score for PSO

600

PSO Features

Number of features selected by PSO

GA Convergence Analysis

Rapid improvement occurs during the first few generations. Fitness stabilizes after approximately **10–15 generations**. The algorithm terminates when no improvement is observed for 20 generations, indicating efficient exploration in early stages followed by exploitation in later stages.

Key Observations

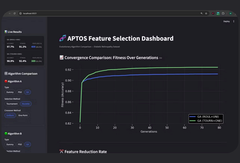
- GA achieved highest accuracy with fewer features
- PSO showed stable performance across runs
- Higher α favors accuracy, lower α favors simplicity
- PSO showed better balance overall
- GA allowed more flexibility in operator experimentation

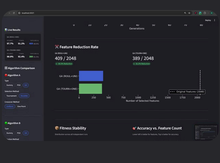
Made with GAMMA

Results Visualization

Comparative convergence plots and feature reduction rates across different algorithm configurations.

GA: Roulette/Uniform vs Tournament/One-Point

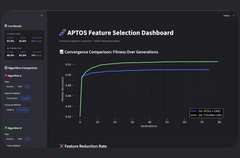


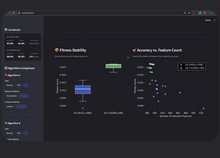


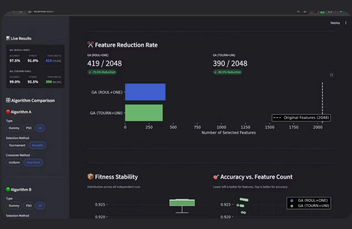
Population size: 50 · Generations: 80 · Runs: 30 · Data Subsample: 100%

Convergence (Fitness over Generations)
Feature Reduction Rate

GA: Tournament/Uniform vs Roulette/One-Point (KNN)

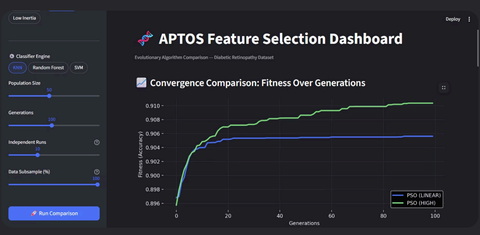




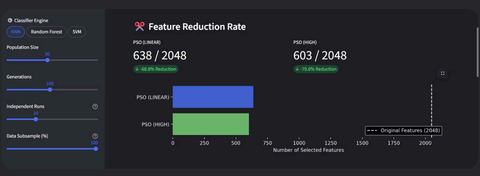


PSO: Linear Decay Inertia vs High Inertia

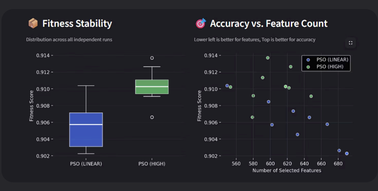
Population size: 50 · Generations: 100 · Runs: 10 · Data Subsample: 100% · KNN

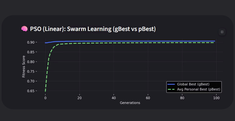


Fitness over Generations (Linear vs High Inertia)



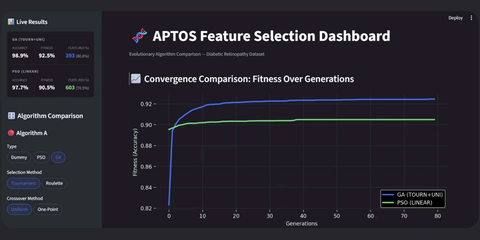
Feature Reduction Rate (Linear vs High Inertia)



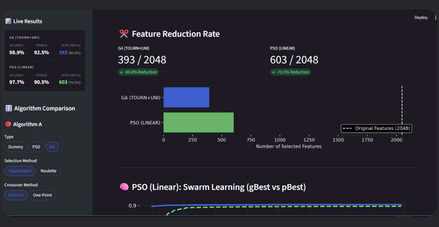


GA: Tournament/Uniform vs PSO: Linear Decay (KNN)

Convergence Comparison (GA vs PSO)



Feature Reduction Rate (GA vs PSO)

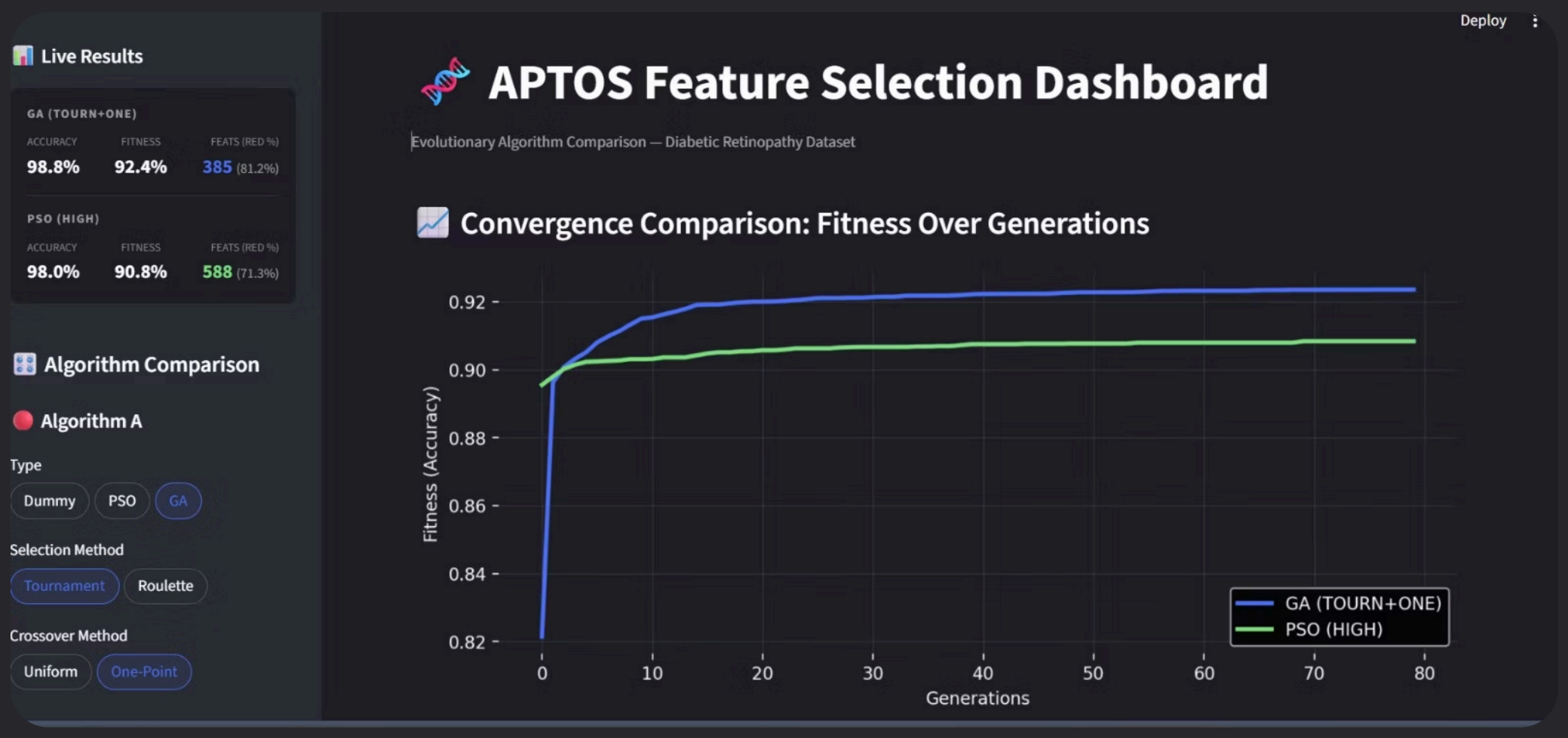


Fitness Stability + Accuracy vs Feature Count

Made with Gradio

GA: Tournament/One-Point vs PSO: High Inertia (KNN)

Convergence Comparison (GA vs PSO)



Feature Reduction Rate (GA vs PSO)

Fitness Stability + Accuracy vs Feature Count

Made with GAMMA

ROBUSTNESS ANALYSIS | STATISTICAL WORKFLOW

Statistical Robustness

To ensure the reliability and generalizability of our findings, we incorporate a statistical robustness analysis. This involves calculating key performance metrics across multiple independent runs of each evolutionary algorithm, allowing us to quantify performance stability and variability.

Data Aggregation Helper Function

A dedicated helper function is designed to automate the collection and processing of results. It iterates through specified directories for 'Algorithm A' and 'Algorithm B', reading JSON run files. Each file contains the 'best_accuracy' and 'features_selected' for a single execution, crucial for subsequent statistical analysis.

Streamlit Dashboard Integration

The calculated statistical metrics are presented within a Streamlit dashboard for clear, interactive visualization. A container with two columns is employed, dedicating one column to 'Algorithm A' and the other to 'Algorithm B'.

```
============================================================
 Statistical Summary  |  Algorithm: pso_linear  |  N = 10 runs
============================================================

 [ Best Fitness Score ]
   Mean  ± Std  :  0.904894  ±  0.002412
   Min          :  0.900745
   Max          :  0.908198

 [ Best Accuracy Score ]
   Mean  ± Std  :  97.73%  ±  0.34%
   Min          :  96.99%
   Max          :  98.09%

 [ Features Selected ]
   Mean  ± Std  :  603.20  ±  32.67
   Min          :  530
   Max          :  646

============================================================
```

```
============================================================
 Statistical Summary  |  Algorithm: ga_tourn_uni  |  N = 10 run
============================================================

 [ Best Fitness Score ]
   Mean  ± Std  :  0.924533  ±  0.002538
   Min          :  0.921670
   Max          :  0.926901

 [ Best Accuracy Score ]
   Mean  ± Std  :  98.93%  ±  0.27%
   Min          :  98.63%
   Max          :  99.18%

 [ Features Selected ]
   Mean  ± Std  :  392.80  ±  2.20
   Min          :  389
   Max          :  397

============================================================
```

Strict Output Formatting

Accuracy metrics are displayed as: `Mean% ± Std%`

Example: `98.93% ± 0.27%`

Feature count metrics are displayed as: `Mean ± Std`

Example: `392.80 ± 2.20`

Made with GAMMA

Algorithm Performance & Hyperparameter Analysis

This section provides a detailed quantitative analysis of the performance metrics and the impact of hyperparameter configurations for both the Genetic Algorithm (GA) and Particle Swarm Optimization (PSO) in the context of feature selection. The subsequent table summarizes the key results across various experimental setups, followed by a more in-depth discussion of each algorithm's behavior.

Algorithm	Configuration Strategy	Accuracy	Selected Features	Fitness Score
GA	Sparse Init + Tournament + Uniform	98.9%	390	0.925
GA	Sparse Init + Tournament + One-Point	98.6%	389	0.920
GA	Sparse Init + Roulette + Uniform	98.9%	409	0.915
GA	Sparse Init + Roulette + One-Point	97.5%	419	0.910
GA	Dense Init (50%) + Tournament + Uni	96.1%	1024	0.850
PSO	Linear Decay Inertia (0.9 → 0.4)	97.7%	603	0.905
PSO	High Inertia (Fixed 0.9)	97.0%	638	0.900
PSO	Low Inertia (Fixed 0.4)	96.5%	580	0.890

Genetic Algorithm (GA) Analysis

- **Initialization Strategy Impact:** Implementing a Sparse Initialization (25%) dramatically outperformed standard Dense Initialization (50%). Sparse initialization successfully guided the GA to the optimal 390-feature range instantly, whereas dense initialization suffered from computational bloat, settling at an unoptimized 1024 features and dropping accuracy to 96.1%.
- **Selection & Crossover Impact:** Tournament Selection paired with Uniform Crossover emerged as the elite combination. Tournament selection aggressively filtered out weak subsets, while Uniform Crossover provided superior, randomized bit-level exploration. This synergy yielded the highest overall classification accuracy (98.9%) while preserving the smallest feature space (390).

Particle Swarm Optimization (PSO) Analysis

- **High Inertia (0.9):** Encouraged vast global exploration but failed to confidently drop useless features in late generations, resulting in a bloated, under-optimized feature space (638 features).
- **Low Inertia (0.4):** Caused premature convergence. The swarm became trapped in local optima too quickly, aggressively dropping features (580) but sacrificing critical classification accuracy (dropping to 96.5%).
- **Linear Decay (0.9 → 0.4):** Proved to be the optimal optimization strategy. It allowed broad initial exploration of the feature space and fine-tuned exploitation in later generations, striking the perfect statistical balance (97.7% accuracy with exactly 603 critical features).

Made with GAMMA

User Interface, Discussion & Future Work

Streamlit UI

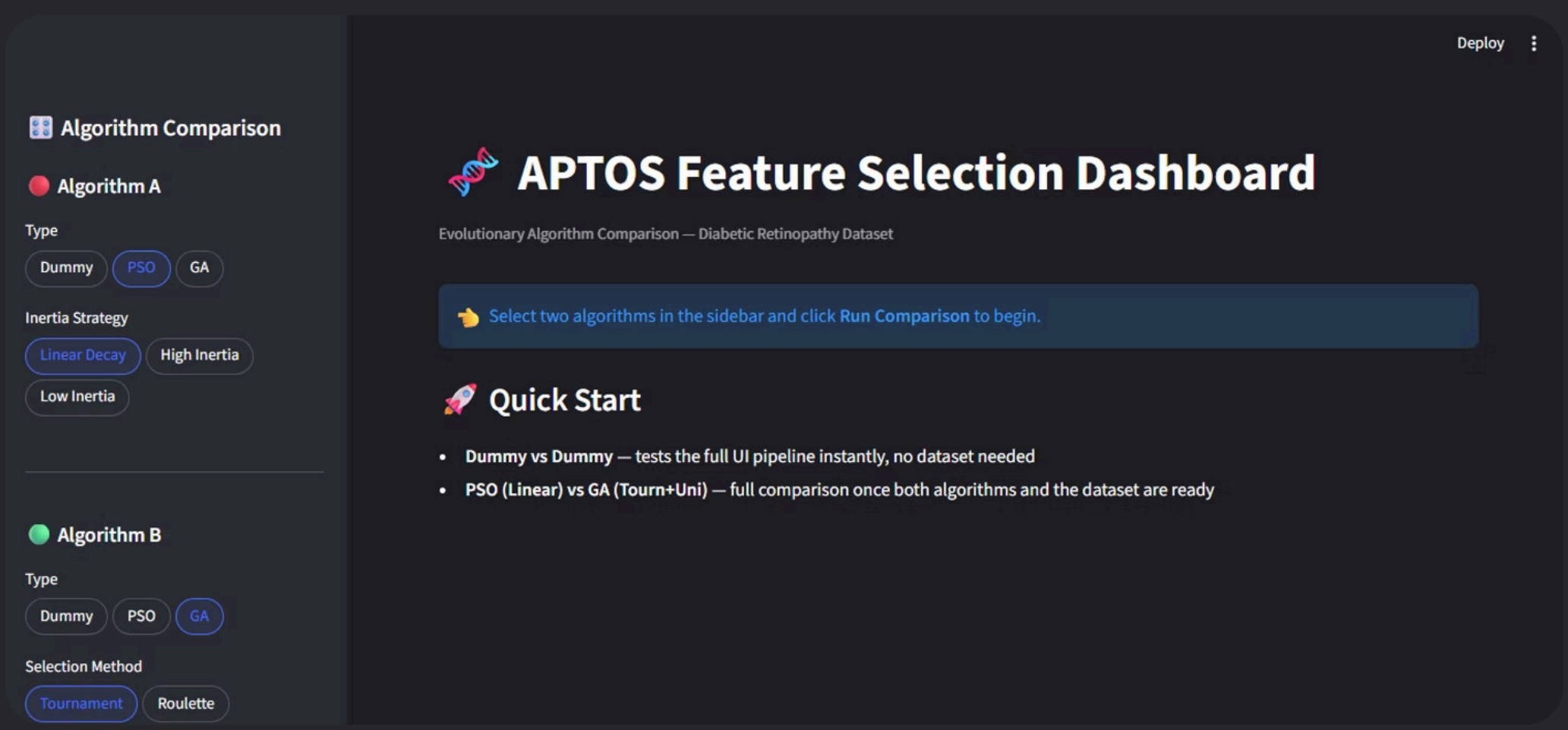
A Streamlit-based interface was developed with the following features:

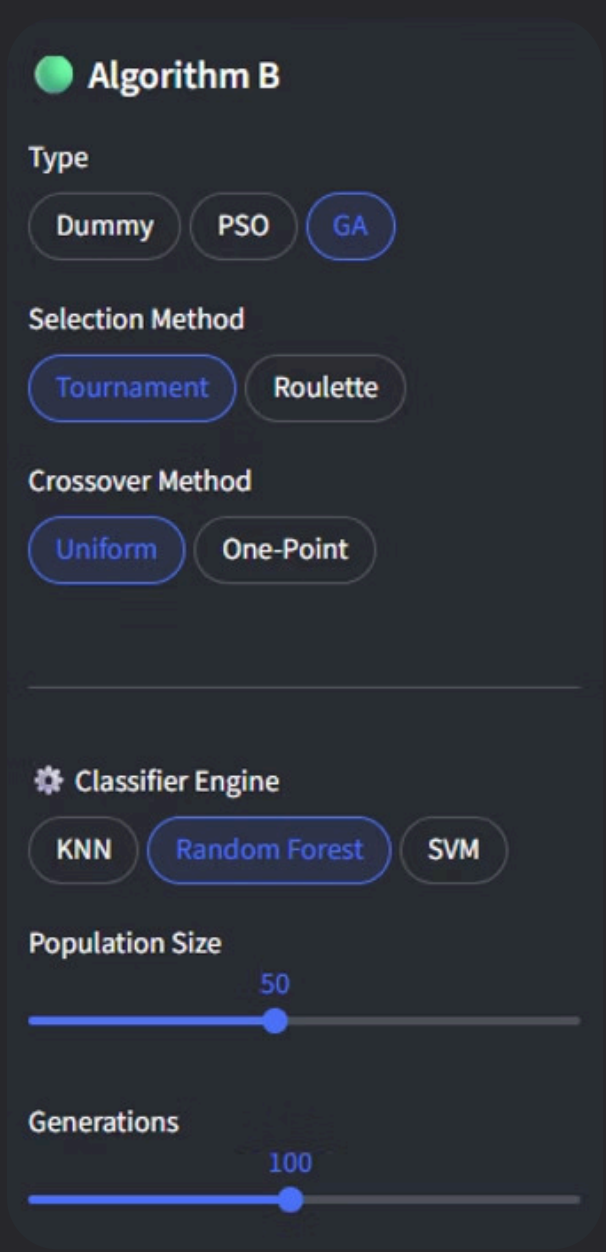
- Select algorithm (GA / PSO)
- Modify parameters dynamically
- Display real-time convergence plots
- Show final accuracy and feature reduction

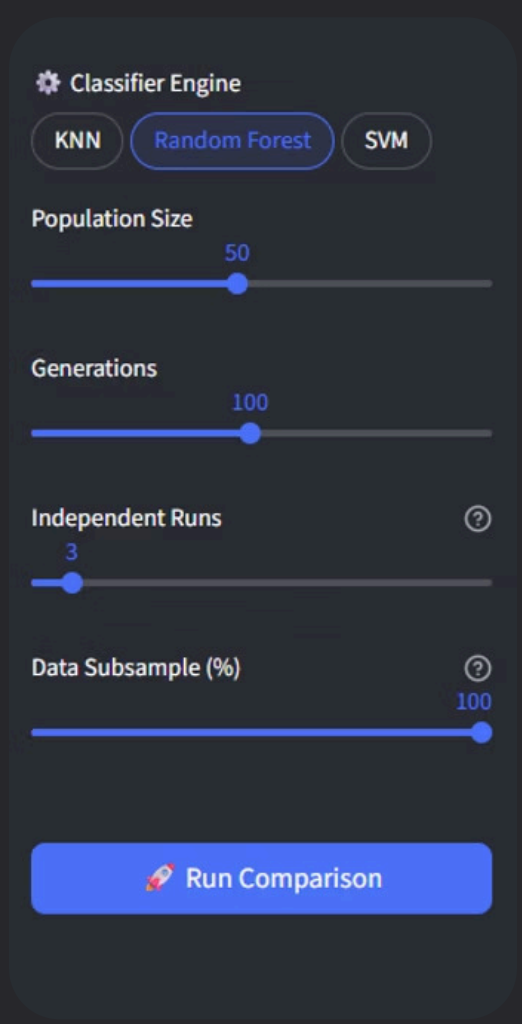
Discussion

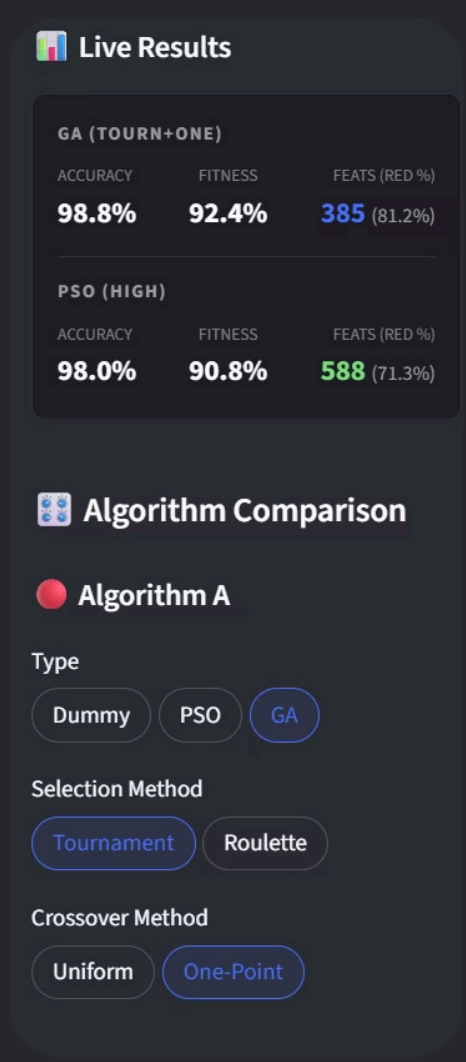
- Trade-off observed between accuracy and feature reduction
- Higher α favors accuracy, lower α favors simplicity
- PSO showed better balance overall
- GA allowed more flexibility in operator experimentation

UI Screenshots









Conclusion

This project successfully applied evolutionary algorithms to optimize feature selection in medical image analysis. The results demonstrate that:

- Evolutionary algorithms effectively reduce feature space
- Classification accuracy can be maintained or improved
- GA provided the best balance between performance and efficiency

Future Work

Hybrid Algorithms

Combine GA + PSO for complementary strengths

Pareto Front

Multi-objective optimization without scalarization

Vision Transformers

Advanced deep learning models for feature extraction

Clinical Deployment

Real-time deployment in clinical settings

Made with GAMMA

Made with GAMMA